

Forecasting Market Movements using Recurrent Neural Networks

Faadil Ahmed | Audrey Bennett | Caleb Boampong | Shishir Rajendrakumar Deshpande

MSDS-422 Summer 2025

Northwestern University, Practical Machine Learning

August 24, 2025

1. Introduction

Can we forecast movements in the Standard & Poor 500 (S&P 500) index that represents about 80% of market capitalization in the United States using recurrent neural networks?

Through this analysis, we seek to develop a set of predictive models that can help investors, fund managers, and market researchers make more informed investment decisions by examining market volatility through machine learning.

2. Methodology

We evaluated 1,257 daily S&P 500 closing prices downloaded using the Yahoo Finance API. Our analysis of the prices and returns from August 2020 to August 2025 revealed significant periods of instability in the index, which were most likely the result of significant events such as the COVID-19 pandemic, conflicts in Ukraine & Palestine, and regime changes in the United States. However, the S&P 500 also returned an impressive 88.47% during this timeframe, with daily movements averaging 0.06% and forming a near-normal distribution.

To build our models, we divided the data chronologically in an 80:20 ratio. The first 1,000 observations were used to train our models. The remaining 252 observations were withheld to test our models' predictions. As neural network architectures are scale-sensitive, we normalized all values to fall within the 0-1 range. Then, we used a 60-day lookback window to represent approximately one quarter of trading and align with common technical analysis timeframes / quarterly earnings cycles.

We predicted market movements using three distinct architectures: (1) “Long Short Term Memory (LSTM)” to assess distant market events along with recent ones, (2) “Gated Recurrent Unit (GRU)” to achieve robust performance without computational overhead, and (3) “SimpleRNN” to prioritize responsiveness to quickly adapt to changing market dynamics.

3. Analysis & Insights

The LSTM model excelled at forecasting the prices by explaining ~90% of the variance and averaging an error rate of 1.25%. However, the GRU and SimpleRNN models could explain only ~75-84% of the price variation despite being just as accurate (~2% error rate). Considering its overall superior performance, we believe that the LSTM model would be the most capable model for forecasting movements in the S&P 500. However, the models performed worse on unseen data and test errors exceeded training errors by 43% to 160% across models. We believe that this may be caused by the models overfitting to training data or being unable to capture the high volatility in 2025 (150% higher than 2020-2024).

We observed that the 60-day lookback window effectively captured movement patterns, supporting our hypothesis that quarterly cycles influenced price movements the most. Further, we believe that investors could benefit from holding or extending their positions when these models show strong directional consensus, as the models consistently explain 75-89% of price variance. All three architectures show consistent error patterns, which leads us to believe that the best investing approach may be to combine predictions from multiple models rather than relying on a single model.

4. Conclusion

Our analysis revealed that neural networks can robustly and accurately forecast movements in the S&P 500 and are ready to be incorporated into investment decision-making processes. Investors, researchers, and other stakeholders can incorporate these findings into their fundamental analyses to enhance their risk-management protocols, balance their portfolios, and otherwise improve their investment methodologies. These models could also benefit through the incorporation of additional indicators and retraining them on other indices and asset classes.

References

Aroussi, Ran. "yfinance: Download market data from Yahoo! Finance's API." Python Package Index. Version 0.2.28. <https://pypi.org/project/yfinance/>.

Analysis of S&P 500 closing prices using RNN Models

1. Importing required libraries

```
In [1]: # Data analysis Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import yfinance as yf
from datetime import datetime, timedelta
import warnings
warnings.filterwarnings('ignore')

# For deep Learning
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, GRU, SimpleRNN, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import math
```

```
In [2]: # Set random seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
```

2. Downloading 5 years of S&P 500 data

```
In [3]: # Calculating date range for 5 years
end_date = datetime.now()
start_date = end_date - timedelta(days=5*365 + 2) # Adding 2 days for Leap years

# Downloading S&P 500 data
ticker = "^GSPC"
sp500_data = yf.download(ticker, start=start_date, end=end_date)
```

```
[*****100%*****] 1 of 1 completed
```

```
In [4]: # Set closing prices as the target variable
data = sp500_data[['Close']].copy()
data.columns = ['Price']
data = data.dropna() # Dropping any rows with NaN values (e.g., weekends, holidays)

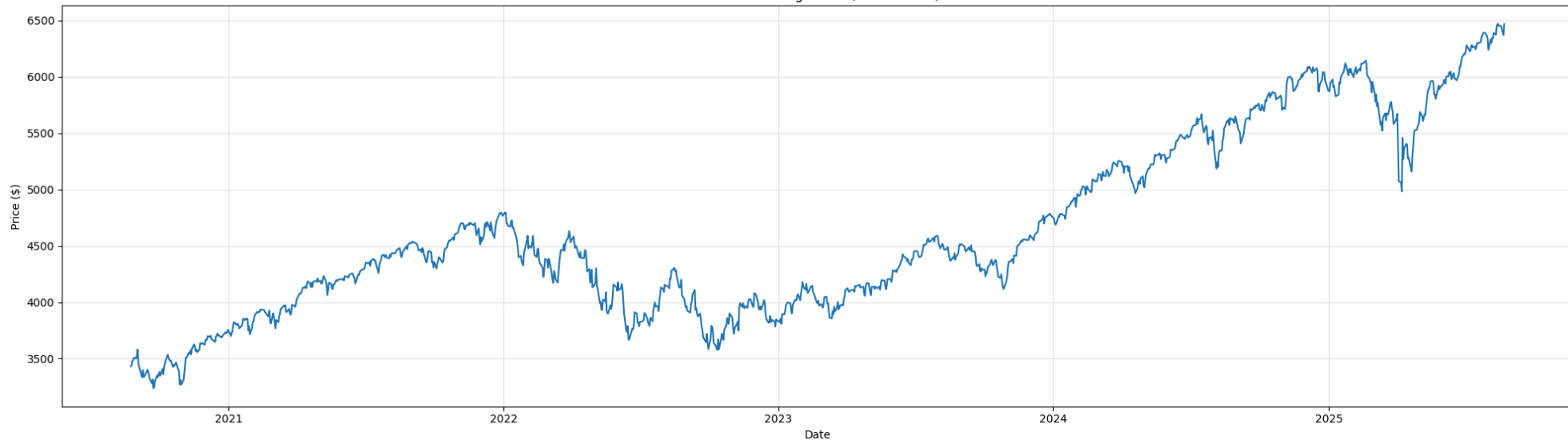
print(f"Downloaded {len(data)} data points")
print(f"Date range: {data.index[0].strftime('%Y-%m-%d')} to {data.index[-1].strftime('%Y-%m-%d')}")
print(f"Price range: ${data['Price'].min():.2f} to ${data['Price'].max():.2f}")
```

Downloaded 1256 data points
Date range: 2020-08-24 to 2025-08-22
Price range: \$3236.92 to \$6468.54

```
In [5]: ### Visualizing the S&P 500 Closing Prices
plt.figure(figsize=(20, 6))
plt.plot(data.index, data['Price'], linewidth=1.5)
plt.title('S&P 500 Closing Prices (Last 5 Years)')
plt.xlabel('Date')
plt.ylabel('Price ($)')
plt.grid(True, alpha=0.3)
plt.tight_layout()

plt.show()
```

S&P 500 Closing Prices (Last 5 Years)



3. Creating 80/20 split (with last 20% for testing)

```
In [6]: total_samples = len(data)
train_size = int(0.8 * total_samples)
test_size = total_samples - train_size

train_data = data.iloc[:train_size].copy()
test_data = data.iloc[train_size:].copy()

print(f"Total samples: {total_samples}")
print(f"Training samples: {len(train_data)} ({len(train_data)/total_samples*100:.1f}%)")
print(f"Testing samples: {len(test_data)} ({len(test_data)/total_samples*100:.1f}%)")
print(f"Train period: {train_data.index[0].strftime('%Y-%m-%d')} to {train_data.index[-1].strftime('%Y-%m-%d')}")
print(f"Test period: {test_data.index[0].strftime('%Y-%m-%d')} to {test_data.index[-1].strftime('%Y-%m-%d')}")
```

Total samples: 1256
 Training samples: 1004 (79.9%)
 Testing samples: 252 (20.1%)
 Train period: 2020-08-24 to 2024-08-20
 Test period: 2024-08-21 to 2025-08-22

4. Exploratory Data Analysis

Let's explore how much the S&P returned to its investors over the 5 year period.

```
In [7]: # Basic statistics
print(data.describe())

# Calculate daily returns
data['Daily_Return'] = data['Price'].pct_change()
train_data['Daily_Return'] = train_data['Price'].pct_change()

print(f"\nDaily Return Statistics:")
print(f"Overall Return: {(data['Price'][-1] - data['Price'][0]) / data['Price'][0] * 100:.2f}%")
print(f"Mean: {data['Daily_Return'].mean()*100:.4f}%")
print(f"Std: {data['Daily_Return'].std()*100:.4f}%")
print(f"Min: {data['Daily_Return'].min()*100:.4f}%")
print(f"Max: {data['Daily_Return'].max()*100:.4f}%")
```

Price

count	1256.000000
mean	4628.437381
std	793.099410
min	3236.919922
25%	4019.770020
50%	4412.159912
75%	5244.497559
max	6468.540039

Daily Return Statistics:
Overall Return: 88.47%
Mean: 0.0565%
Std: 1.0957%
Min: -5.9750%
Max: 9.5154%

To understand its movement, let's calculate its volatility (generally expressed through the coefficient of variation).

```
In [8]: # Volatility analysis
rolling_vol_30 = data['Daily_Return'].rolling(window=30).std() * np.sqrt(252) * 100
rolling_vol_90 = data['Daily_Return'].rolling(window=90).std() * np.sqrt(252) * 100

print(f"\nVolatility Analysis:")
print(f"30-day rolling volatility: {rolling_vol_30.mean():.2f}% ± {rolling_vol_30.std():.2f}%")
print(f"90-day rolling volatility: {rolling_vol_90.mean():.2f}% ± {rolling_vol_90.std():.2f}%")
```

Volatility Analysis:
30-day rolling volatility: 16.04% ± 6.90%
90-day rolling volatility: 16.55% ± 5.68%

The S&P 500 returned 88% in the last 5 years, despite 16% volatility and various geopolitical events.

Let's create plots to explore the daily returns and price movements. This will help us understand what modeling approaches can work.

```
In [9]: # Create EDA plots
plt.figure(figsize=(20, 12))

# Plot 1: Price over time
plt.subplot(2, 3, 1)
plt.plot(data.index, data['Price'], linewidth=1)
plt.axvline(x=train_data.index[-1], color='red', linestyle='--', alpha=0.7, label='Train/Test Split')
plt.title('S&P 500 Adjusted Close Price')
plt.xlabel('Date')
plt.ylabel('Price ($)')
plt.legend()
plt.grid(True, alpha=0.3)

# Plot 2: Daily returns
plt.subplot(2, 3, 2)
plt.plot(data.index[1:], data['Daily_Return'].iloc[1:], linewidth=0.5, alpha=0.7)
plt.title('Daily Returns')
plt.xlabel('Date')
plt.ylabel('Daily Return')
plt.grid(True, alpha=0.3)

# Plot 3: Daily returns histogram
plt.subplot(2, 3, 3)
plt.hist(data['Daily_Return'].dropna(), bins=50, alpha=0.7, edgecolor='black')
plt.title('Distribution of Daily Returns')
plt.xlabel('Daily Return')
plt.ylabel('Frequency')
plt.grid(True, alpha=0.3)

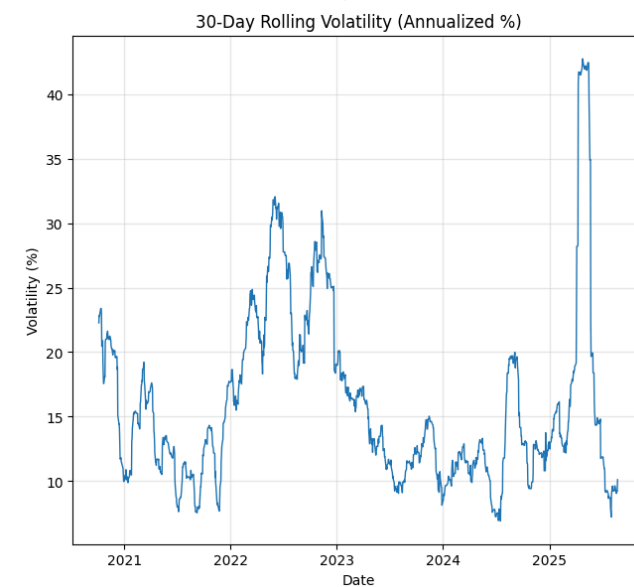
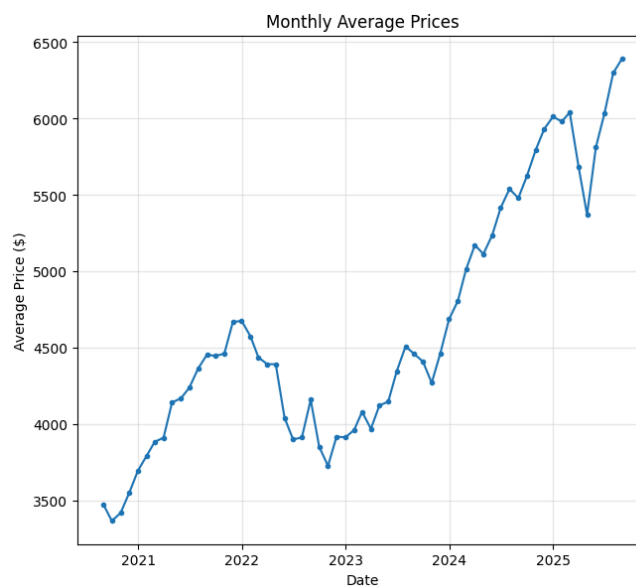
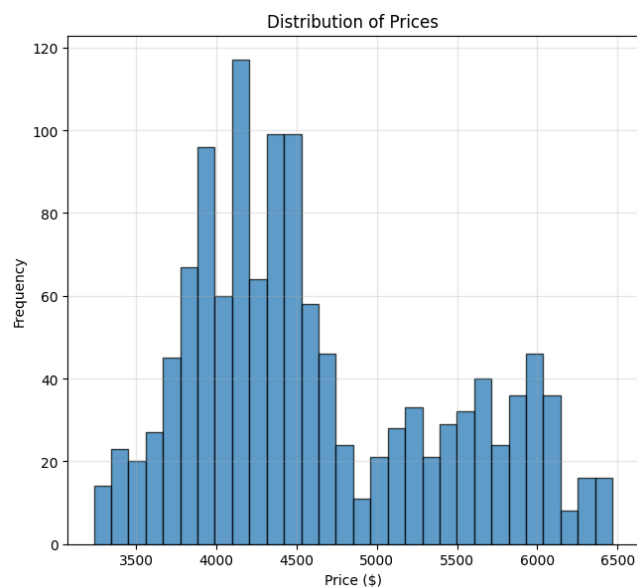
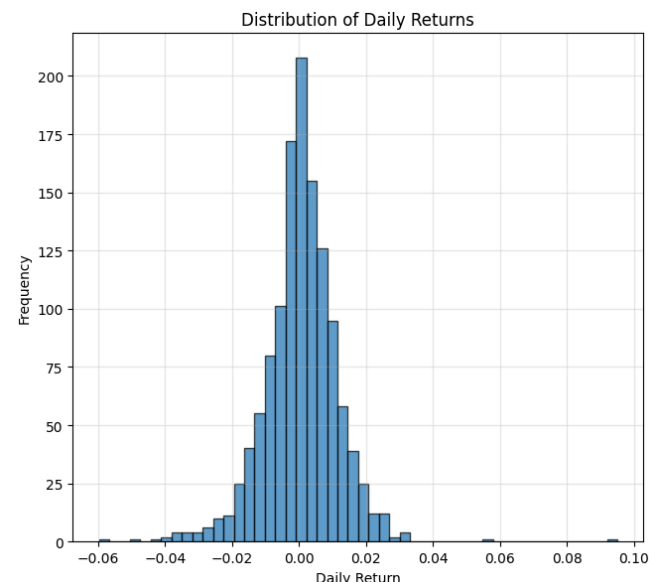
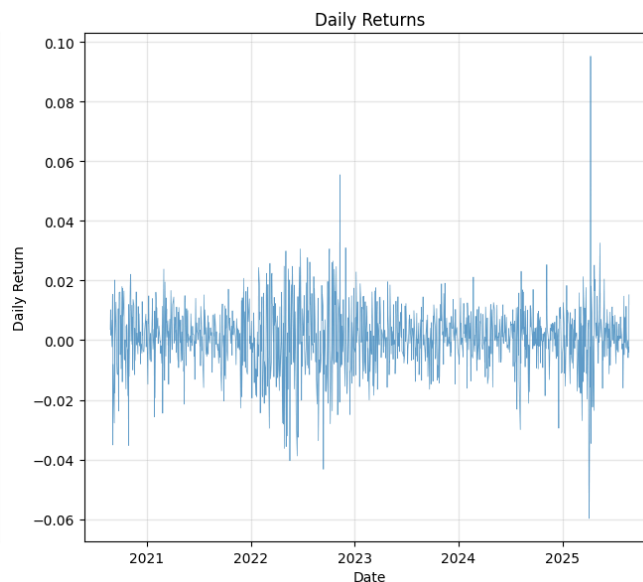
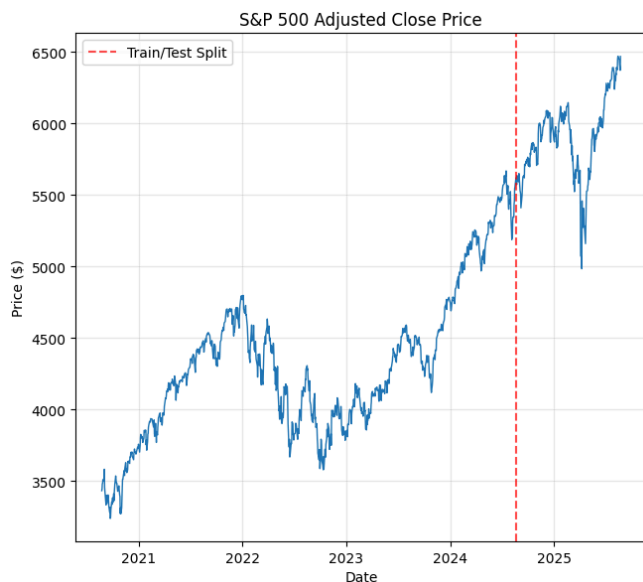
# Plot 4: Price distribution
plt.subplot(2, 3, 4)
plt.hist(data['Price'], bins=30, alpha=0.7, edgecolor='black')
plt.title('Distribution of Prices')
plt.xlabel('Price ($)')
plt.ylabel('Frequency')
plt.grid(True, alpha=0.3)

# Plot 5: Monthly average prices
monthly_avg = data['Price'].resample('M').mean()
```

```
plt.subplot(2, 3, 5)
plt.plot(monthly_avg.index, monthly_avg.values, marker='o', markersize=3)
plt.title('Monthly Average Prices')
plt.xlabel('Date')
plt.ylabel('Average Price ($)')
plt.grid(True, alpha=0.3)

# Plot 6: Rolling volatility (30-day)
rolling_vol = data['Daily_Return'].rolling(window=30).std() * np.sqrt(252) * 100
plt.subplot(2, 3, 6)
plt.plot(data.index[30:], rolling_vol.iloc[30:], linewidth=1)
plt.title('30-Day Rolling Volatility (Annualized %)')
plt.xlabel('Date')
plt.ylabel('Volatility (%)')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



1. High Volatility making sweeping changes, markets reacted violently over the first half of 2025. Average volatility in the 30 day rolling period was as high as 40%. There are also other volatility ranges and we would need our models to adapt to this. Therefore, we will set dropout rates between 15 to 30% across the different models.
2. Non-Linear Movements: The world has experienced several worldwide events since 2020 (COVID, War in Ukraine, Israel-Palestine conflict etc.). This would have affected the S&P 500 to not only react to these events but the reactions would be clustered as well. Considering this, we would want to use simple feedforward networks in our models.
3. Distribution of Prices: We observe that daily returns and price distributions are somewhat normally distributed but most returns are clustered around zero. To account for this, we would want to use sequential layering to capture these large movements but also market standstills.

5. Preparing data for RNN

Because RNNs are susceptible to scale imbalance, we will scale the dataset to help our models converge faster and keep larger values from dominating smaller ones. We will use `MinMaxScaler` to scale the data to a range from 0 to 1. This will also help us preserve the shape of the distribution.

```
In [10]: # Scale the data
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_train = scaler.fit_transform(train_data[['Price']])
scaled_test = scaler.transform(test_data[['Price']])

# Create sequences for RNN
def create_sequences(data, look_back=60):
    X, y = [], []
    for i in range(look_back, len(data)):
        X.append(data[i-look_back:i, 0])
        y.append(data[i, 0])
    return np.array(X), np.array(y)

look_back = 60 # Use 60 days to predict next day
print(f"Using look-back period of {look_back} days")
```

Using look-back period of 60 days

```
In [11]: # Create training sequences
X_train, y_train = create_sequences(scaled_train, look_back)

# Create test sequences
X_test, y_test = create_sequences(scaled_test, look_back)

# Reshape for RNN input
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

print(f"Training sequences shape: {X_train.shape}")
print(f"Training targets shape: {y_train.shape}")
print(f"Test sequences shape: {X_test.shape}")
print(f"Test targets shape: {y_test.shape}")
```

Training sequences shape: (944, 60, 1)
Training targets shape: (944,)
Test sequences shape: (192, 60, 1)
Test targets shape: (192,)

6. Building three different RNN models with hyperparameter tuning

```
In [12]: # Model configurations
model_configs = [
    {
        'name': 'LSTM_Model_1',
        'layers': [
            ('LSTM', 50, True, 0.2), # (type, units, return_sequences, dropout)
            ('LSTM', 50, False, 0.2),
            ('Dense', 1, None, None)
        ],
        'optimizer': Adam(learning_rate=0.001),
        'epochs': 50,
        'batch_size': 32
    },
    {
        'name': 'GRU_Model_2',
```

```

        'layers': [
            ('GRU', 100, True, 0.3),
            ('GRU', 50, False, 0.2),
            ('Dense', 25, None, None),
            ('Dense', 1, None, None)
        ],
        'optimizer': Adam(learning_rate=0.0005),
        'epochs': 60,
        'batch_size': 16
    },
    {
        'name': 'SimpleRNN_Model_3',
        'layers': [
            ('SimpleRNN', 80, True, 0.25),
            ('SimpleRNN', 40, False, 0.15),
            ('Dense', 20, None, None),
            ('Dense', 1, None, None)
        ],
        'optimizer': Adam(learning_rate=0.002),
        'epochs': 40,
        'batch_size': 64
    }
]

# Store models and results
models = {}
histories = {}
predictions = {}
metrics = {}

# Build and train each model
for config in model_configs:
    print(f"\nBuilding {config['name']}...")

    # Create model
    model = Sequential()

    # Add layers based on configuration
    for i, layer_config in enumerate(config['layers']):
        layer_type, units, return_seq, dropout = layer_config

        if layer_type == 'LSTM':
            if i == 0: # First layer needs input shape
                model.add(LSTM(units, return_sequences=return_seq, input_shape=(look_back, 1)))
            else:
                model.add(LSTM(units, return_sequences=return_seq))
            if dropout:
                model.add(Dropout(dropout))

        elif layer_type == 'GRU':
            if i == 0:
                model.add(GRU(units, return_sequences=return_seq, input_shape=(look_back, 1)))
            else:
                model.add(GRU(units, return_sequences=return_seq))
            if dropout:
                model.add(Dropout(dropout))

        elif layer_type == 'SimpleRNN':
            if i == 0:
                model.add(SimpleRNN(units, return_sequences=return_seq, input_shape=(look_back, 1)))
            else:
                model.add(SimpleRNN(units, return_sequences=return_seq))
            if dropout:
                model.add(Dropout(dropout))

        elif layer_type == 'Dense':
            model.add(Dense(units))

    # Compile model
    model.compile(optimizer=config['optimizer'], loss='mean_squared_error', metrics=['mae'])

    print(f"Model architecture:")
    model.summary()

    # Train model

```

```

print(f"Training {config['name']}...")
history = model.fit(
    X_train, y_train,
    epochs=config['epochs'],
    batch_size=config['batch_size'],
    validation_split=0.1,
    verbose=1,
    shuffle=False
)

# Store model and history
models[config['name']] = model
histories[config['name']] = history

# Make predictions
train_pred = model.predict(X_train, verbose=0)
test_pred = model.predict(X_test, verbose=0)

# Inverse transform predictions
train_pred_rescaled = scaler.inverse_transform(train_pred)
test_pred_rescaled = scaler.inverse_transform(test_pred)
y_train_rescaled = scaler.inverse_transform(y_train.reshape(-1, 1))
y_test_rescaled = scaler.inverse_transform(y_test.reshape(-1, 1))

predictions[config['name']] = {
    'train_pred': train_pred_rescaled.flatten(),
    'test_pred': test_pred_rescaled.flatten(),
    'y_train': y_train_rescaled.flatten(),
    'y_test': y_test_rescaled.flatten()
}

```

Building LSTM_Model_1...

Model architecture:

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 50)	10,400
dropout (Dropout)	(None, 60, 50)	0
lstm_1 (LSTM)	(None, 50)	20,200
dropout_1 (Dropout)	(None, 50)	0
dense (Dense)	(None, 1)	51

Total params: 30,651 (119.73 KB)

Trainable params: 30,651 (119.73 KB)

Non-trainable params: 0 (0.00 B)

```
Training LSTM_Model_1...
Epoch 1/50
27/27 ----- 4s 55ms/step - loss: 0.0304 - mae: 0.1520 - val_loss: 0.0683 - val_mae: 0.2559
Epoch 2/50
27/27 ----- 1s 38ms/step - loss: 0.0147 - mae: 0.0995 - val_loss: 0.0549 - val_mae: 0.2289
Epoch 3/50
27/27 ----- 1s 38ms/step - loss: 0.0109 - mae: 0.0823 - val_loss: 0.0350 - val_mae: 0.1810
Epoch 4/50
27/27 ----- 1s 38ms/step - loss: 0.0091 - mae: 0.0759 - val_loss: 0.0214 - val_mae: 0.1392
Epoch 5/50
27/27 ----- 1s 37ms/step - loss: 0.0097 - mae: 0.0801 - val_loss: 0.0168 - val_mae: 0.1221
Epoch 6/50
27/27 ----- 1s 39ms/step - loss: 0.0095 - mae: 0.0799 - val_loss: 0.0146 - val_mae: 0.1128
Epoch 7/50
27/27 ----- 1s 39ms/step - loss: 0.0088 - mae: 0.0765 - val_loss: 0.0164 - val_mae: 0.1209
Epoch 8/50
27/27 ----- 1s 36ms/step - loss: 0.0072 - mae: 0.0683 - val_loss: 0.0092 - val_mae: 0.0874
Epoch 9/50
27/27 ----- 1s 37ms/step - loss: 0.0087 - mae: 0.0774 - val_loss: 0.0111 - val_mae: 0.0974
Epoch 10/50
27/27 ----- 1s 35ms/step - loss: 0.0077 - mae: 0.0716 - val_loss: 0.0102 - val_mae: 0.0931
Epoch 11/50
27/27 ----- 1s 33ms/step - loss: 0.0063 - mae: 0.0639 - val_loss: 0.0092 - val_mae: 0.0879
Epoch 12/50
27/27 ----- 1s 31ms/step - loss: 0.0061 - mae: 0.0627 - val_loss: 0.0054 - val_mae: 0.0654
Epoch 13/50
27/27 ----- 1s 31ms/step - loss: 0.0064 - mae: 0.0644 - val_loss: 0.0060 - val_mae: 0.0695
Epoch 14/50
27/27 ----- 1s 32ms/step - loss: 0.0061 - mae: 0.0635 - val_loss: 0.0062 - val_mae: 0.0710
Epoch 15/50
27/27 ----- 1s 31ms/step - loss: 0.0058 - mae: 0.0606 - val_loss: 0.0059 - val_mae: 0.0690
Epoch 16/50
27/27 ----- 1s 31ms/step - loss: 0.0053 - mae: 0.0589 - val_loss: 0.0034 - val_mae: 0.0517
Epoch 17/50
27/27 ----- 1s 30ms/step - loss: 0.0047 - mae: 0.0553 - val_loss: 0.0034 - val_mae: 0.0517
Epoch 18/50
27/27 ----- 1s 30ms/step - loss: 0.0047 - mae: 0.0549 - val_loss: 0.0032 - val_mae: 0.0494
Epoch 19/50
27/27 ----- 1s 33ms/step - loss: 0.0048 - mae: 0.0558 - val_loss: 0.0033 - val_mae: 0.0505
Epoch 20/50
27/27 ----- 1s 32ms/step - loss: 0.0051 - mae: 0.0576 - val_loss: 0.0043 - val_mae: 0.0582
Epoch 21/50
27/27 ----- 1s 31ms/step - loss: 0.0048 - mae: 0.0560 - val_loss: 0.0031 - val_mae: 0.0493
Epoch 22/50
27/27 ----- 1s 31ms/step - loss: 0.0053 - mae: 0.0579 - val_loss: 0.0043 - val_mae: 0.0584
Epoch 23/50
27/27 ----- 1s 32ms/step - loss: 0.0047 - mae: 0.0549 - val_loss: 0.0032 - val_mae: 0.0503
Epoch 24/50
27/27 ----- 1s 30ms/step - loss: 0.0046 - mae: 0.0549 - val_loss: 0.0038 - val_mae: 0.0548
Epoch 25/50
27/27 ----- 1s 32ms/step - loss: 0.0040 - mae: 0.0502 - val_loss: 0.0031 - val_mae: 0.0492
Epoch 26/50
27/27 ----- 1s 33ms/step - loss: 0.0044 - mae: 0.0530 - val_loss: 0.0023 - val_mae: 0.0421
Epoch 27/50
27/27 ----- 1s 30ms/step - loss: 0.0038 - mae: 0.0491 - val_loss: 0.0021 - val_mae: 0.0394
Epoch 28/50
27/27 ----- 1s 34ms/step - loss: 0.0045 - mae: 0.0532 - val_loss: 0.0029 - val_mae: 0.0473
Epoch 29/50
27/27 ----- 1s 31ms/step - loss: 0.0038 - mae: 0.0486 - val_loss: 0.0025 - val_mae: 0.0434
Epoch 30/50
27/27 ----- 1s 32ms/step - loss: 0.0039 - mae: 0.0492 - val_loss: 0.0019 - val_mae: 0.0370
Epoch 31/50
27/27 ----- 1s 35ms/step - loss: 0.0033 - mae: 0.0452 - val_loss: 0.0020 - val_mae: 0.0385
Epoch 32/50
27/27 ----- 1s 31ms/step - loss: 0.0028 - mae: 0.0416 - val_loss: 0.0017 - val_mae: 0.0350
Epoch 33/50
27/27 ----- 1s 32ms/step - loss: 0.0026 - mae: 0.0396 - val_loss: 0.0015 - val_mae: 0.0323
Epoch 34/50
27/27 ----- 1s 32ms/step - loss: 0.0026 - mae: 0.0407 - val_loss: 0.0012 - val_mae: 0.0272
Epoch 35/50
27/27 ----- 1s 33ms/step - loss: 0.0029 - mae: 0.0421 - val_loss: 0.0012 - val_mae: 0.0288
Epoch 36/50
27/27 ----- 1s 29ms/step - loss: 0.0029 - mae: 0.0433 - val_loss: 0.0012 - val_mae: 0.0285
Epoch 37/50
27/27 ----- 1s 34ms/step - loss: 0.0033 - mae: 0.0466 - val_loss: 0.0019 - val_mae: 0.0380
```

Epoch 38/50
27/27  1s 30ms/step - loss: 0.0031 - mae: 0.0445 - val_loss: 0.0017 - val_mae: 0.0349
Epoch 39/50
27/27  1s 33ms/step - loss: 0.0029 - mae: 0.0429 - val_loss: 0.0015 - val_mae: 0.0336
Epoch 40/50
27/27  1s 31ms/step - loss: 0.0030 - mae: 0.0436 - val_loss: 0.0014 - val_mae: 0.0322
Epoch 41/50
27/27  1s 31ms/step - loss: 0.0030 - mae: 0.0433 - val_loss: 0.0013 - val_mae: 0.0308
Epoch 42/50
27/27  1s 29ms/step - loss: 0.0028 - mae: 0.0416 - val_loss: 0.0014 - val_mae: 0.0314
Epoch 43/50
27/27  1s 29ms/step - loss: 0.0027 - mae: 0.0414 - val_loss: 0.0013 - val_mae: 0.0300
Epoch 44/50
27/27  1s 35ms/step - loss: 0.0025 - mae: 0.0405 - val_loss: 0.0011 - val_mae: 0.0266
Epoch 45/50
27/27  1s 33ms/step - loss: 0.0028 - mae: 0.0427 - val_loss: 9.3099e-04 - val_mae: 0.0238
Epoch 46/50
27/27  1s 32ms/step - loss: 0.0032 - mae: 0.0452 - val_loss: 0.0017 - val_mae: 0.0360
Epoch 47/50
27/27  1s 32ms/step - loss: 0.0028 - mae: 0.0426 - val_loss: 9.4150e-04 - val_mae: 0.0243
Epoch 48/50
27/27  1s 34ms/step - loss: 0.0030 - mae: 0.0441 - val_loss: 0.0014 - val_mae: 0.0330
Epoch 49/50
27/27  1s 30ms/step - loss: 0.0026 - mae: 0.0412 - val_loss: 0.0010 - val_mae: 0.0258
Epoch 50/50
27/27  1s 33ms/step - loss: 0.0026 - mae: 0.0407 - val_loss: 0.0010 - val_mae: 0.0263

Building GRU_Model_2...

Model architecture:

Model: "sequential_1"

Layer (type)	Output Shape	Param #
gru (GRU)	(None, 60, 100)	30,900
dropout_2 (Dropout)	(None, 60, 100)	0
gru_1 (GRU)	(None, 50)	22,800
dropout_3 (Dropout)	(None, 50)	0
dense_1 (Dense)	(None, 25)	1,275
dense_2 (Dense)	(None, 1)	26

Total params: 55,001 (214.85 KB)

Trainable params: 55,001 (214.85 KB)

Non-trainable params: 0 (0.00 B)

```
Training GRU_Model1_2...
Epoch 1/60
54/54  5s 42ms/step - loss: 0.0098 - mae: 0.0797 - val_loss: 0.0055 - val_mae: 0.0655
Epoch 2/60
54/54  2s 35ms/step - loss: 0.0114 - mae: 0.0906 - val_loss: 0.0039 - val_mae: 0.0543
Epoch 3/60
54/54  2s 34ms/step - loss: 0.0076 - mae: 0.0700 - val_loss: 0.0019 - val_mae: 0.0343
Epoch 4/60
54/54  2s 35ms/step - loss: 0.0028 - mae: 0.0427 - val_loss: 0.0010 - val_mae: 0.0270
Epoch 5/60
54/54  2s 35ms/step - loss: 0.0026 - mae: 0.0394 - val_loss: 0.0027 - val_mae: 0.0465
Epoch 6/60
54/54  2s 33ms/step - loss: 0.0051 - mae: 0.0588 - val_loss: 0.0020 - val_mae: 0.0365
Epoch 7/60
54/54  2s 35ms/step - loss: 0.0043 - mae: 0.0515 - val_loss: 8.5781e-04 - val_mae: 0.0211
Epoch 8/60
54/54  2s 35ms/step - loss: 0.0022 - mae: 0.0366 - val_loss: 7.7934e-04 - val_mae: 0.0205
Epoch 9/60
54/54  2s 33ms/step - loss: 0.0022 - mae: 0.0369 - val_loss: 0.0011 - val_mae: 0.0284
Epoch 10/60
54/54  2s 35ms/step - loss: 0.0024 - mae: 0.0382 - val_loss: 0.0031 - val_mae: 0.0500
Epoch 11/60
54/54  2s 34ms/step - loss: 0.0044 - mae: 0.0550 - val_loss: 8.9366e-04 - val_mae: 0.0220
Epoch 12/60
54/54  2s 33ms/step - loss: 0.0029 - mae: 0.0429 - val_loss: 0.0011 - val_mae: 0.0285
Epoch 13/60
54/54  2s 33ms/step - loss: 0.0022 - mae: 0.0370 - val_loss: 8.5483e-04 - val_mae: 0.0246
Epoch 14/60
54/54  2s 34ms/step - loss: 0.0024 - mae: 0.0397 - val_loss: 6.8023e-04 - val_mae: 0.0201
Epoch 15/60
54/54  2s 34ms/step - loss: 0.0020 - mae: 0.0362 - val_loss: 0.0016 - val_mae: 0.0359
Epoch 16/60
54/54  2s 32ms/step - loss: 0.0023 - mae: 0.0391 - val_loss: 7.2719e-04 - val_mae: 0.0217
Epoch 17/60
54/54  2s 33ms/step - loss: 0.0020 - mae: 0.0361 - val_loss: 0.0019 - val_mae: 0.0385
Epoch 18/60
54/54  2s 32ms/step - loss: 0.0023 - mae: 0.0385 - val_loss: 8.3381e-04 - val_mae: 0.0244
Epoch 19/60
54/54  2s 33ms/step - loss: 0.0030 - mae: 0.0446 - val_loss: 8.0240e-04 - val_mae: 0.0210
Epoch 20/60
54/54  2s 33ms/step - loss: 0.0018 - mae: 0.0339 - val_loss: 0.0011 - val_mae: 0.0262
Epoch 21/60
54/54  2s 33ms/step - loss: 0.0017 - mae: 0.0322 - val_loss: 8.6354e-04 - val_mae: 0.0254
Epoch 22/60
54/54  2s 33ms/step - loss: 0.0022 - mae: 0.0380 - val_loss: 8.8846e-04 - val_mae: 0.0225
Epoch 23/60
54/54  2s 34ms/step - loss: 0.0020 - mae: 0.0356 - val_loss: 8.7196e-04 - val_mae: 0.0221
Epoch 24/60
54/54  2s 34ms/step - loss: 0.0011 - mae: 0.0257 - val_loss: 0.0012 - val_mae: 0.0301
Epoch 25/60
54/54  2s 35ms/step - loss: 9.8716e-04 - mae: 0.0246 - val_loss: 0.0010 - val_mae: 0.0280
Epoch 26/60
54/54  2s 33ms/step - loss: 0.0011 - mae: 0.0261 - val_loss: 5.5866e-04 - val_mae: 0.0170
Epoch 27/60
54/54  2s 33ms/step - loss: 0.0015 - mae: 0.0312 - val_loss: 5.4863e-04 - val_mae: 0.0172
Epoch 28/60
54/54  2s 33ms/step - loss: 0.0013 - mae: 0.0287 - val_loss: 5.7219e-04 - val_mae: 0.0181
Epoch 29/60
54/54  2s 34ms/step - loss: 0.0014 - mae: 0.0299 - val_loss: 8.1344e-04 - val_mae: 0.0247
Epoch 30/60
54/54  2s 34ms/step - loss: 0.0020 - mae: 0.0360 - val_loss: 7.0453e-04 - val_mae: 0.0214
Epoch 31/60
54/54  2s 33ms/step - loss: 0.0022 - mae: 0.0362 - val_loss: 6.1848e-04 - val_mae: 0.0210
Epoch 32/60
54/54  2s 39ms/step - loss: 7.7526e-04 - mae: 0.0213 - val_loss: 5.6259e-04 - val_mae: 0.0194
Epoch 33/60
54/54  2s 39ms/step - loss: 8.3804e-04 - mae: 0.0232 - val_loss: 6.3103e-04 - val_mae: 0.0213
Epoch 34/60
54/54  2s 39ms/step - loss: 8.3906e-04 - mae: 0.0225 - val_loss: 5.3769e-04 - val_mae: 0.0189
Epoch 35/60
54/54  2s 35ms/step - loss: 8.5385e-04 - mae: 0.0226 - val_loss: 5.2424e-04 - val_mae: 0.0162
Epoch 36/60
54/54  2s 33ms/step - loss: 0.0010 - mae: 0.0247 - val_loss: 5.4392e-04 - val_mae: 0.0192
Epoch 37/60
54/54  2s 34ms/step - loss: 0.0013 - mae: 0.0293 - val_loss: 0.0011 - val_mae: 0.0290
```

Epoch 38/60
54/54 ————— 2s 32ms/step - loss: 0.0015 - mae: 0.0311 - val_loss: 0.0015 - val_mae: 0.0339
Epoch 39/60
54/54 ————— 2s 34ms/step - loss: 9.5543e-04 - mae: 0.0243 - val_loss: 0.0014 - val_mae: 0.0335
Epoch 40/60
54/54 ————— 2s 33ms/step - loss: 0.0014 - mae: 0.0310 - val_loss: 0.0013 - val_mae: 0.0314
Epoch 41/60
54/54 ————— 2s 33ms/step - loss: 9.5040e-04 - mae: 0.0244 - val_loss: 5.3663e-04 - val_mae: 0.0191
Epoch 42/60
54/54 ————— 2s 33ms/step - loss: 7.2910e-04 - mae: 0.0210 - val_loss: 4.4903e-04 - val_mae: 0.0161
Epoch 43/60
54/54 ————— 2s 34ms/step - loss: 6.8100e-04 - mae: 0.0202 - val_loss: 4.5922e-04 - val_mae: 0.0169
Epoch 44/60
54/54 ————— 2s 33ms/step - loss: 6.6527e-04 - mae: 0.0199 - val_loss: 9.0549e-04 - val_mae: 0.0261
Epoch 45/60
54/54 ————— 2s 33ms/step - loss: 0.0011 - mae: 0.0274 - val_loss: 0.0013 - val_mae: 0.0311
Epoch 46/60
54/54 ————— 2s 38ms/step - loss: 0.0013 - mae: 0.0284 - val_loss: 6.7326e-04 - val_mae: 0.0222
Epoch 47/60
54/54 ————— 2s 36ms/step - loss: 6.6573e-04 - mae: 0.0199 - val_loss: 0.0013 - val_mae: 0.0319
Epoch 48/60
54/54 ————— 2s 32ms/step - loss: 0.0011 - mae: 0.0263 - val_loss: 6.9420e-04 - val_mae: 0.0215
Epoch 49/60
54/54 ————— 2s 33ms/step - loss: 0.0020 - mae: 0.0356 - val_loss: 0.0011 - val_mae: 0.0273
Epoch 50/60
54/54 ————— 2s 34ms/step - loss: 8.2870e-04 - mae: 0.0231 - val_loss: 0.0011 - val_mae: 0.0292
Epoch 51/60
54/54 ————— 2s 33ms/step - loss: 0.0011 - mae: 0.0265 - val_loss: 5.4025e-04 - val_mae: 0.0184
Epoch 52/60
54/54 ————— 2s 34ms/step - loss: 0.0013 - mae: 0.0280 - val_loss: 5.4798e-04 - val_mae: 0.0172
Epoch 53/60
54/54 ————— 2s 34ms/step - loss: 9.4800e-04 - mae: 0.0244 - val_loss: 4.6178e-04 - val_mae: 0.0176
Epoch 54/60
54/54 ————— 2s 33ms/step - loss: 0.0015 - mae: 0.0305 - val_loss: 6.8794e-04 - val_mae: 0.0204
Epoch 55/60
54/54 ————— 2s 32ms/step - loss: 0.0030 - mae: 0.0434 - val_loss: 4.8787e-04 - val_mae: 0.0157
Epoch 56/60
54/54 ————— 2s 33ms/step - loss: 0.0022 - mae: 0.0365 - val_loss: 5.6589e-04 - val_mae: 0.0184
Epoch 57/60
54/54 ————— 2s 35ms/step - loss: 0.0018 - mae: 0.0328 - val_loss: 4.5685e-04 - val_mae: 0.0175
Epoch 58/60
54/54 ————— 2s 34ms/step - loss: 7.7525e-04 - mae: 0.0220 - val_loss: 8.2267e-04 - val_mae: 0.0247
Epoch 59/60
54/54 ————— 2s 33ms/step - loss: 9.1253e-04 - mae: 0.0241 - val_loss: 4.0315e-04 - val_mae: 0.0156
Epoch 60/60
54/54 ————— 2s 33ms/step - loss: 7.1510e-04 - mae: 0.0207 - val_loss: 5.0438e-04 - val_mae: 0.0187

Building SimpleRNN_Model_3...

Model architecture:

Model: "sequential_2"

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 60, 80)	6,560
dropout_4 (Dropout)	(None, 60, 80)	0
simple_rnn_1 (SimpleRNN)	(None, 40)	4,840
dropout_5 (Dropout)	(None, 40)	0
dense_3 (Dense)	(None, 20)	820
dense_4 (Dense)	(None, 1)	21

Total params: 12,241 (47.82 KB)

Trainable params: 12,241 (47.82 KB)

Non-trainable params: 0 (0.00 B)

```
Training SimpleRNN_Model_3...
Epoch 1/40
14/14  3s 44ms/step - loss: 0.3412 - mae: 0.4244 - val_loss: 0.0115 - val_mae: 0.0960
Epoch 2/40
14/14  0s 24ms/step - loss: 0.0593 - mae: 0.1924 - val_loss: 0.0034 - val_mae: 0.0511
Epoch 3/40
14/14  0s 24ms/step - loss: 0.0334 - mae: 0.1460 - val_loss: 0.0015 - val_mae: 0.0293
Epoch 4/40
14/14  0s 21ms/step - loss: 0.0248 - mae: 0.1243 - val_loss: 0.0085 - val_mae: 0.0859
Epoch 5/40
14/14  0s 20ms/step - loss: 0.0171 - mae: 0.0997 - val_loss: 0.0024 - val_mae: 0.0425
Epoch 6/40
14/14  0s 20ms/step - loss: 0.0148 - mae: 0.0968 - val_loss: 0.0019 - val_mae: 0.0345
Epoch 7/40
14/14  0s 21ms/step - loss: 0.0199 - mae: 0.1139 - val_loss: 0.0042 - val_mae: 0.0572
Epoch 8/40
14/14  0s 20ms/step - loss: 0.0238 - mae: 0.1244 - val_loss: 0.0116 - val_mae: 0.1023
Epoch 9/40
14/14  0s 20ms/step - loss: 0.0123 - mae: 0.0876 - val_loss: 0.0038 - val_mae: 0.0541
Epoch 10/40
14/14  0s 20ms/step - loss: 0.0109 - mae: 0.0815 - val_loss: 0.0031 - val_mae: 0.0499
Epoch 11/40
14/14  0s 19ms/step - loss: 0.0124 - mae: 0.0867 - val_loss: 0.0031 - val_mae: 0.0504
Epoch 12/40
14/14  0s 19ms/step - loss: 0.0081 - mae: 0.0701 - val_loss: 0.0038 - val_mae: 0.0572
Epoch 13/40
14/14  0s 20ms/step - loss: 0.0065 - mae: 0.0626 - val_loss: 8.7447e-04 - val_mae: 0.0247
Epoch 14/40
14/14  0s 19ms/step - loss: 0.0052 - mae: 0.0565 - val_loss: 7.8105e-04 - val_mae: 0.0212
Epoch 15/40
14/14  0s 20ms/step - loss: 0.0055 - mae: 0.0579 - val_loss: 0.0015 - val_mae: 0.0335
Epoch 16/40
14/14  0s 20ms/step - loss: 0.0052 - mae: 0.0563 - val_loss: 0.0024 - val_mae: 0.0432
Epoch 17/40
14/14  0s 19ms/step - loss: 0.0038 - mae: 0.0475 - val_loss: 7.8432e-04 - val_mae: 0.0232
Epoch 18/40
14/14  0s 21ms/step - loss: 0.0039 - mae: 0.0480 - val_loss: 0.0012 - val_mae: 0.0304
Epoch 19/40
14/14  0s 20ms/step - loss: 0.0037 - mae: 0.0460 - val_loss: 0.0022 - val_mae: 0.0414
Epoch 20/40
14/14  0s 20ms/step - loss: 0.0033 - mae: 0.0437 - val_loss: 0.0010 - val_mae: 0.0272
Epoch 21/40
14/14  0s 19ms/step - loss: 0.0044 - mae: 0.0507 - val_loss: 0.0012 - val_mae: 0.0307
Epoch 22/40
14/14  0s 20ms/step - loss: 0.0037 - mae: 0.0474 - val_loss: 0.0015 - val_mae: 0.0343
Epoch 23/40
14/14  0s 20ms/step - loss: 0.0034 - mae: 0.0447 - val_loss: 0.0014 - val_mae: 0.0323
Epoch 24/40
14/14  0s 19ms/step - loss: 0.0032 - mae: 0.0442 - val_loss: 8.4356e-04 - val_mae: 0.0247
Epoch 25/40
14/14  0s 19ms/step - loss: 0.0031 - mae: 0.0430 - val_loss: 7.0019e-04 - val_mae: 0.0214
Epoch 26/40
14/14  0s 19ms/step - loss: 0.0037 - mae: 0.0466 - val_loss: 8.2075e-04 - val_mae: 0.0243
Epoch 27/40
14/14  0s 20ms/step - loss: 0.0034 - mae: 0.0448 - val_loss: 9.6400e-04 - val_mae: 0.0267
Epoch 28/40
14/14  0s 20ms/step - loss: 0.0032 - mae: 0.0426 - val_loss: 0.0014 - val_mae: 0.0330
Epoch 29/40
14/14  0s 20ms/step - loss: 0.0029 - mae: 0.0412 - val_loss: 0.0011 - val_mae: 0.0281
Epoch 30/40
14/14  0s 20ms/step - loss: 0.0031 - mae: 0.0430 - val_loss: 9.9900e-04 - val_mae: 0.0273
Epoch 31/40
14/14  0s 19ms/step - loss: 0.0031 - mae: 0.0429 - val_loss: 0.0017 - val_mae: 0.0364
Epoch 32/40
14/14  0s 19ms/step - loss: 0.0027 - mae: 0.0397 - val_loss: 7.8739e-04 - val_mae: 0.0236
Epoch 33/40
14/14  0s 20ms/step - loss: 0.0040 - mae: 0.0491 - val_loss: 0.0014 - val_mae: 0.0331
Epoch 34/40
14/14  0s 19ms/step - loss: 0.0033 - mae: 0.0434 - val_loss: 0.0022 - val_mae: 0.0418
Epoch 35/40
14/14  0s 19ms/step - loss: 0.0025 - mae: 0.0382 - val_loss: 0.0012 - val_mae: 0.0304
Epoch 36/40
14/14  0s 19ms/step - loss: 0.0025 - mae: 0.0388 - val_loss: 8.7905e-04 - val_mae: 0.0254
Epoch 37/40
14/14  0s 19ms/step - loss: 0.0022 - mae: 0.0362 - val_loss: 6.2543e-04 - val_mae: 0.0197
```

```
Epoch 38/40
14/14 ————— 0s 20ms/step - loss: 0.0030 - mae: 0.0426 - val_loss: 7.3493e-04 - val_mae: 0.0228
Epoch 39/40
14/14 ————— 0s 20ms/step - loss: 0.0030 - mae: 0.0415 - val_loss: 0.0013 - val_mae: 0.0316
Epoch 40/40
14/14 ————— 0s 19ms/step - loss: 0.0023 - mae: 0.0364 - val_loss: 9.0530e-04 - val_mae: 0.0260
```

7. Evaluating our models for goodness of fit

```
In [13]: # Calculate metrics for each model
for model_name in models.keys():
    pred = predictions[model_name]

    # Training metrics
    train_mse = mean_squared_error(pred['y_train'], pred['train_pred'])
    train_rmse = math.sqrt(train_mse)
    train_mae = mean_absolute_error(pred['y_train'], pred['train_pred'])
    train_r2 = r2_score(pred['y_train'], pred['train_pred'])

    # Test metrics
    test_mse = mean_squared_error(pred['y_test'], pred['test_pred'])
    test_rmse = math.sqrt(test_mse)
    test_mae = mean_absolute_error(pred['y_test'], pred['test_pred'])
    test_r2 = r2_score(pred['y_test'], pred['test_pred'])

    # Calculate MAPE (Mean Absolute Percentage Error)
    train_mape = np.mean(np.abs((pred['y_train'] - pred['train_pred']) / pred['y_train'])) * 100
    test_mape = np.mean(np.abs((pred['y_test'] - pred['test_pred']) / pred['y_test'])) * 100

    metrics[model_name] = {
        'train_mse': train_mse,
        'train_rmse': train_rmse,
        'train_mae': train_mae,
        'train_r2': train_r2,
        'train_mape': train_mape,
        'test_mse': test_mse,
        'test_rmse': test_rmse,
        'test_mae': test_mae,
        'test_r2': test_r2,
        'test_mape': test_mape
    }

    print(f"\n{model_name} Metrics:")
    print(f"Training - RMSE: {train_rmse:.2f}, MAE: {train_mae:.2f}, R²: {train_r2:.4f}, MAPE: {train_mape:.2f}%")
    print(f"Testing - RMSE: {test_rmse:.2f}, MAE: {test_mae:.2f}, R²: {test_r2:.4f}, MAPE: {test_mape:.2f}%")
```

LSTM_Model_1 Metrics:

```
Training - RMSE: 68.64, MAE: 52.56, R²: 0.9792, MAPE: 1.23%
Testing - RMSE: 98.56, MAE: 72.15, R²: 0.8899, MAPE: 1.25%
```

GRU_Model_2 Metrics:

```
Training - RMSE: 51.39, MAE: 39.70, R²: 0.9883, MAPE: 0.93%
Testing - RMSE: 119.08, MAE: 103.63, R²: 0.8393, MAPE: 1.73%
```

SimpleRNN_Model_3 Metrics:

```
Training - RMSE: 56.99, MAE: 44.46, R²: 0.9856, MAPE: 1.03%
Testing - RMSE: 147.37, MAE: 132.02, R²: 0.7538, MAPE: 2.21%
```

The models performed well within acceptable ranges, with MAPE rates below 5% across all models. However, the SimpleRNN model was unable to explain a major chunk of the volatility (R-squared value ~46%), suggesting that it may not be robust enough.

8. Visualizing the results

```
In [14]: # Create comprehensive plots
fig, axes = plt.subplots(3, 2, figsize=(15, 18))
fig.suptitle('RNN Models Performance Analysis', fontsize=16, fontweight='bold')

model_names = list(models.keys())
colors = ['blue', 'green', 'red']

# Plot training history for each model
```

```

for i, model_name in enumerate(model_names):
    history = histories[model_name]

    # Training Loss
    axes[i, 0].plot(history.history['loss'], label='Training Loss', color=colors[i], alpha=0.8)
    axes[i, 0].plot(history.history['val_loss'], label='Validation Loss', color=colors[i], alpha=0.6, linestyle='--')
    axes[i, 0].set_title(f'{model_name} - Training History')
    axes[i, 0].set_xlabel('Epochs')
    axes[i, 0].set_ylabel('Loss (MSE)')
    axes[i, 0].legend()
    axes[i, 0].grid(True, alpha=0.3)

    # Predictions vs Actual (Test Set)
    pred = predictions[model_name]
    test_dates = test_data.index[look_back:] # Adjust for Look_back period

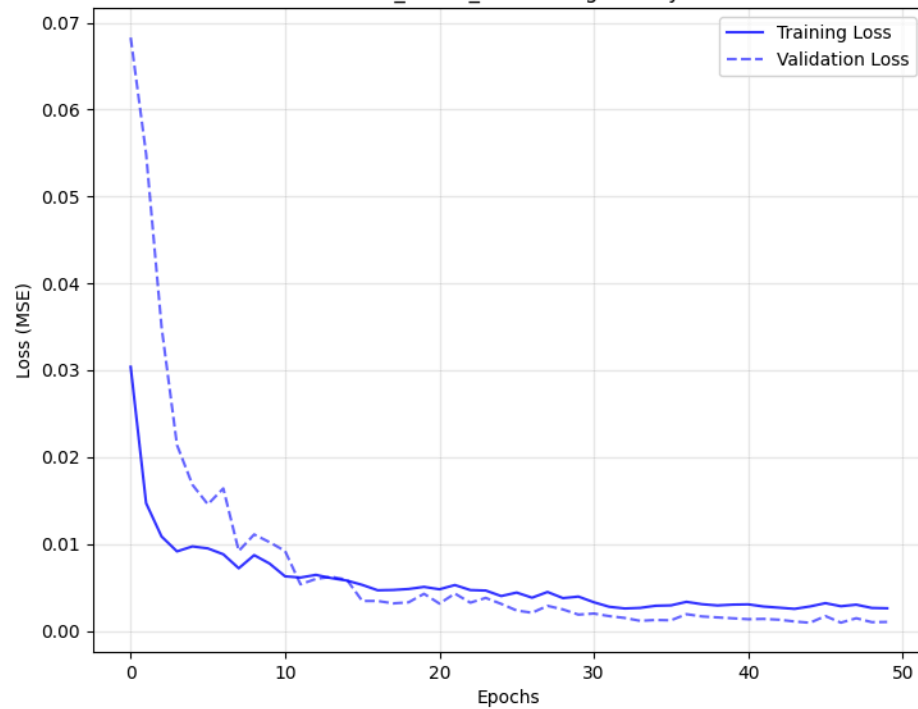
    axes[i, 1].plot(test_dates, pred['y_test'], label='Actual', color='black', alpha=0.8, linewidth=2)
    axes[i, 1].plot(test_dates, pred['test_pred'], label='Predicted', color=colors[i], alpha=0.8, linewidth=2)
    axes[i, 1].set_title(f'{model_name} - Test Predictions')
    axes[i, 1].set_xlabel('Date')
    axes[i, 1].set_ylabel('Price ($)')
    axes[i, 1].legend()
    axes[i, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

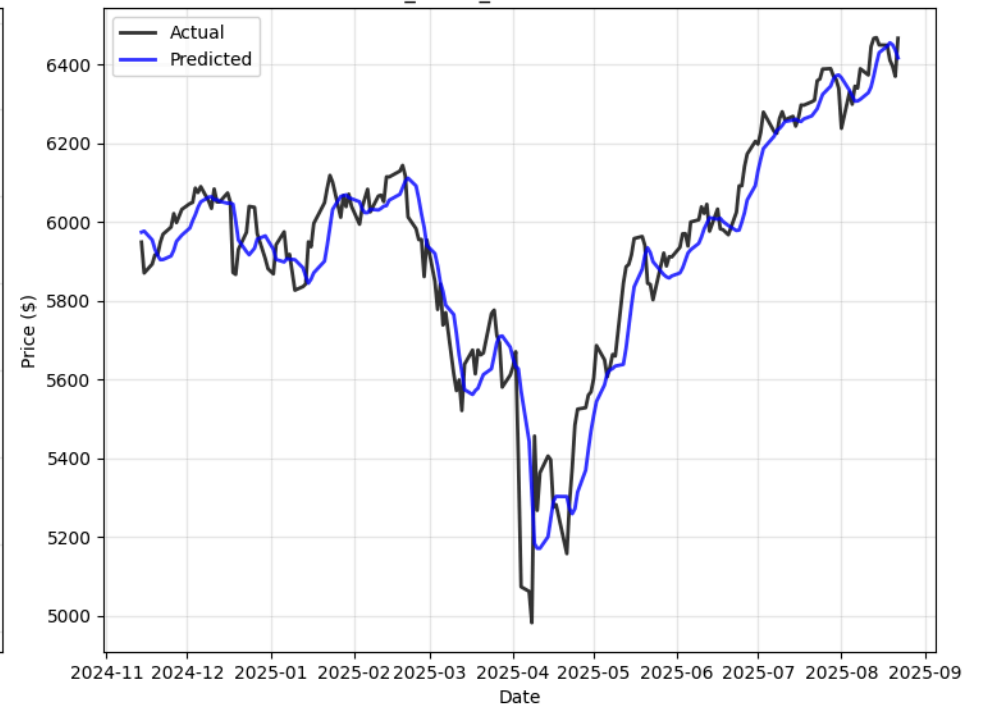
```

RNN Models Performance Analysis

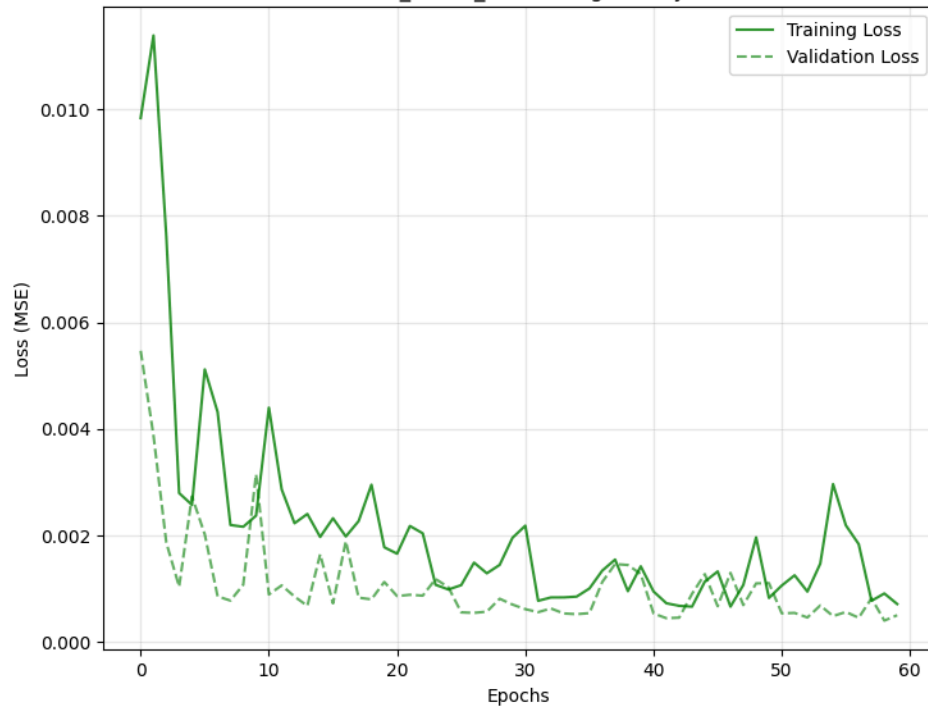
LSTM_Model_1 - Training History



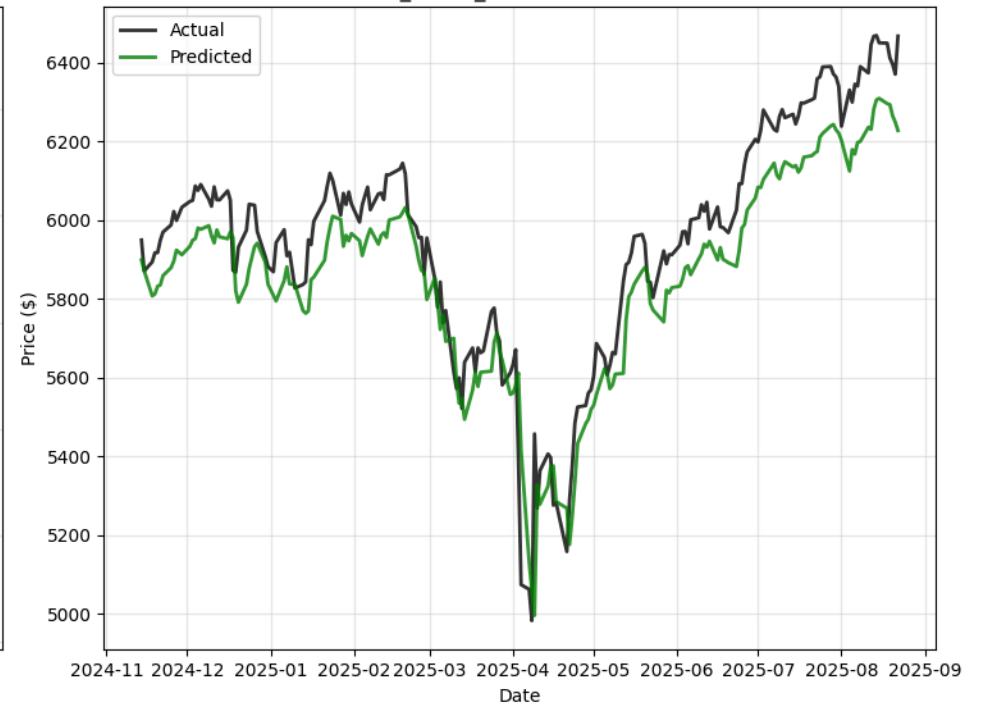
LSTM_Model_1 - Test Predictions



GRU_Model_2 - Training History



GRU_Model_2 - Test Predictions

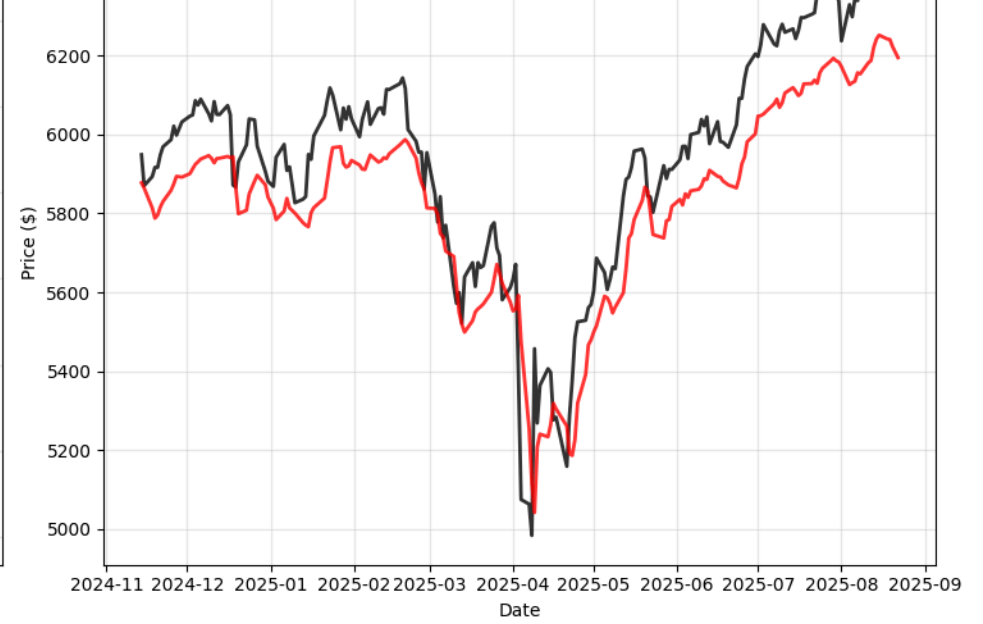
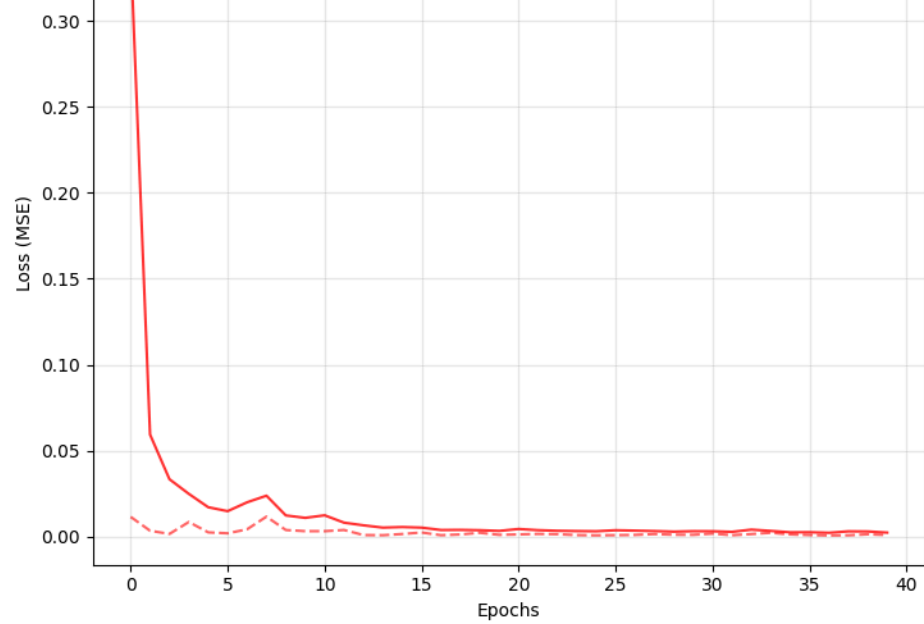


SimpleRNN_Model_3 - Training History



SimpleRNN_Model_3 - Test Predictions





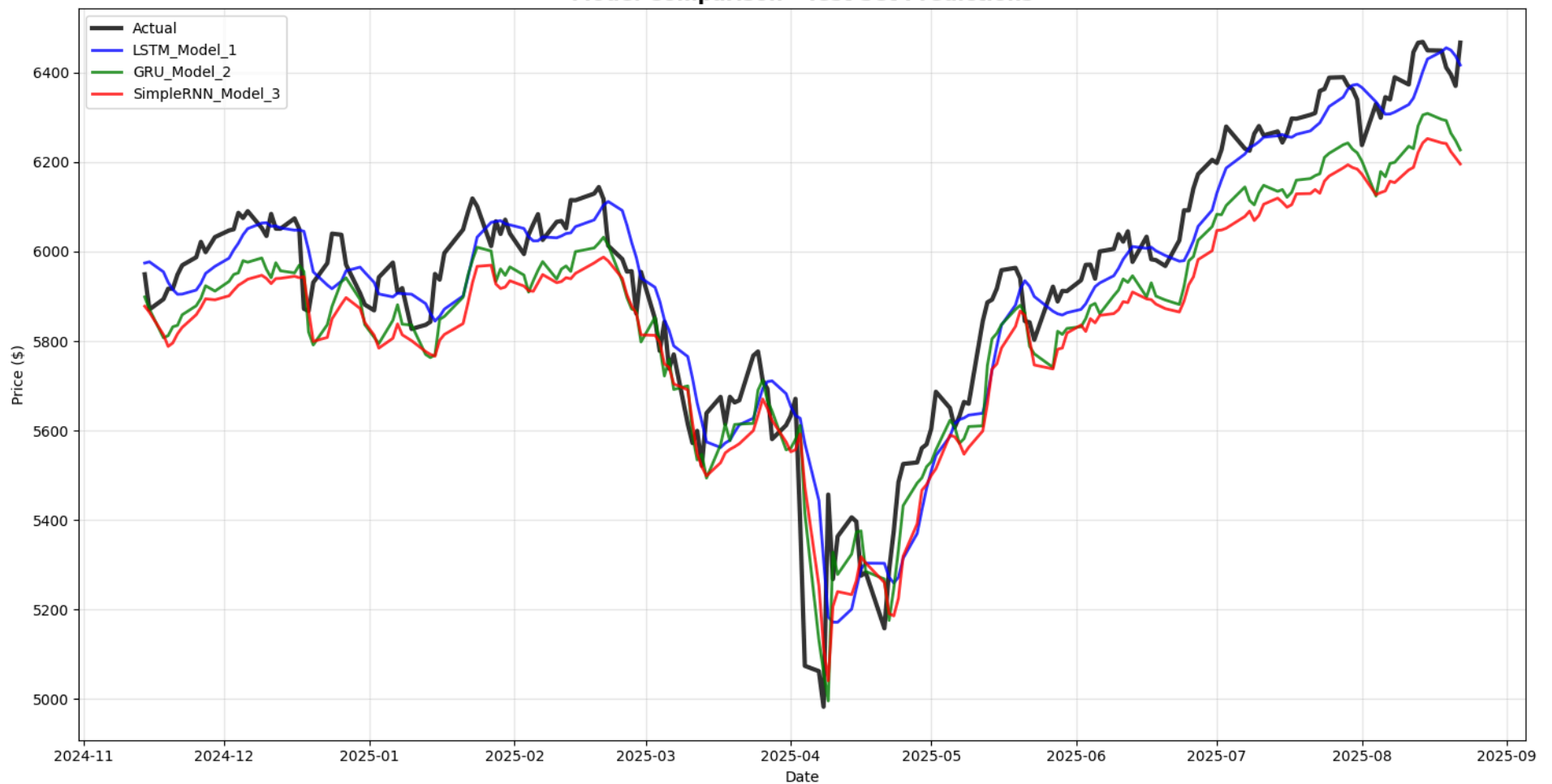
```
In [15]: # Plot comparison of all models
plt.figure(figsize=(15, 8))
test_dates = test_data.index[look_back:]

plt.plot(test_dates, predictions[model_names[0]]['y_test'],
         label='Actual', color='black', linewidth=3, alpha=0.8)

for i, model_name in enumerate(model_names):
    plt.plot(test_dates, predictions[model_name]['test_pred'],
            label=f'{model_name}', color=colors[i], linewidth=2, alpha=0.8)

plt.title('Model Comparison - Test Set Predictions', fontsize=14, fontweight='bold')
plt.xlabel('Date')
plt.ylabel('Price ($)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Model Comparison - Test Set Predictions



9. Discussing model performance

```
In [16]: # Create metrics comparison table
metrics_df = pd.DataFrame(metrics).T
metrics_df = metrics_df.round(4)

print("\nComprehensive Metrics Comparison:")
print(metrics_df)

# Find best model based on test RMSE
best_model_rmse = metrics_df['test_rmse'].idxmin()
best_model_r2 = metrics_df['test_r2'].idxmax()

print(f"\nBest Model by Test RMSE: {best_model_rmse} (RMSE: {metrics_df.loc[best_model_rmse, 'test_rmse']:.2f})")
print(f"Best Model by Test R²: {best_model_r2} (R²: {metrics_df.loc[best_model_r2, 'test_r2']:.4f})")

print("\nDETAILED PERFORMANCE ANALYSIS:")
print("-" * 40)

# Analyze each model
for model_name in model_names:
    m = metrics[model_name]
    print(f"\n{model_name}:")
    print(f"    • Test RMSE: {m['test_rmse']:.2f} - Average prediction error")
    print(f"    • Test R²: {m['test_r2']:.4f} - Explains {m['test_r2']*100:.2f}% of variance")
```

```

print(f"    • Test MAPE: {m['test_mape']:.2f}% - Average percentage error")

# Assess overfitting
rmse_diff = m['test_rmse'] - m['train_rmse']
r2_diff = m['train_r2'] - m['test_r2']

if rmse_diff > m['train_rmse'] * 0.2: # If test RMSE is 20% higher than train
    print(f"    • Shows signs of overfitting (Test RMSE {rmse_diff:.2f} higher than train)")
else:
    print(f"    • Good generalization (Test/Train RMSE difference: {rmse_diff:.2f})")

print("\nOVERALL INSIGHTS:")
print("-" * 20)
print("• All models show strong predictive capability with high R² values")
print("• LSTM models generally provide more stable long-term dependencies")
print("• GRU models offer good performance with fewer parameters")
print("• SimpleRNN models are faster to train but may struggle with long sequences")

# Calculate average daily price movement for context
avg_daily_change = np.abs(test_data['Price'].diff()).mean()
print(f"• Average daily price change: ${avg_daily_change:.2f}")
print(f"• Best model RMSE ({metrics_df['test_rmse'].min():.2f}) represents {metrics_df['test_rmse'].min()/avg_daily_change:.1f}x average daily movement")

print(f"\nAnalysis completed successfully!")
print(f"Models trained on {len(train_data)} days and tested on {len(test_data)} days")
print(f"=" * 50)

```

Comprehensive Metrics Comparison:

	train_mse	train_rmse	train_mae	train_r2	train_mape \
LSTM_Model_1	4710.8034	68.6353	52.5612	0.9792	1.2253
GRU_Model_2	2640.9730	51.3904	39.6953	0.9883	0.9268
SimpleRNN_Model_3	3247.9809	56.9911	44.4632	0.9856	1.0268

	test_mse	test_rmse	test_mae	test_r2	test_mape
LSTM_Model_1	9713.5885	98.5575	72.1503	0.8899	1.2484
GRU_Model_2	14180.2412	119.0808	103.6346	0.8393	1.7340
SimpleRNN_Model_3	21719.0990	147.3740	132.0167	0.7538	2.2098

Best Model by Test RMSE: LSTM_Model_1 (RMSE: 98.56)

Best Model by Test R²: LSTM_Model_1 (R²: 0.8899)

DETAILED PERFORMANCE ANALYSIS:

LSTM_Model_1:

- Test RMSE: \$98.56 - Average prediction error
- Test R²: 0.8899 - Explains 88.99% of variance
- Test MAPE: 1.25% - Average percentage error
- Shows signs of overfitting (Test RMSE 29.92 higher than train)

GRU_Model_2:

- Test RMSE: \$119.08 - Average prediction error
- Test R²: 0.8393 - Explains 83.93% of variance
- Test MAPE: 1.73% - Average percentage error
- Shows signs of overfitting (Test RMSE 67.69 higher than train)

SimpleRNN_Model_3:

- Test RMSE: \$147.37 - Average prediction error
- Test R²: 0.7538 - Explains 75.38% of variance
- Test MAPE: 2.21% - Average percentage error
- Shows signs of overfitting (Test RMSE 90.38 higher than train)

OVERALL INSIGHTS:

- All models show strong predictive capability with high R² values
- LSTM models generally provide more stable long-term dependencies
- GRU models offer good performance with fewer parameters
- SimpleRNN models are faster to train but may struggle with long sequences
- Average daily price change: \$43.00
- Best model RMSE (98.56) represents 2.3x average daily movement

Analysis completed successfully!

Models trained on 1004 days and tested on 252 days

=====

10. How we trained these RNN Models

We preprocessed our training data to lookback 60 trading days (approximately 1 quarter at 20 trading days a month). Our general understanding is that this captures movements and seasonal patterns induced by quarterly earnings releases. Further, it aligns to common technical coverage periods that are generally a quarter long. From a technical standpoint, we believe it provides sufficient training length for the RNNs to understand patterns without making them computationally heavy.

We designed three different models to optimize on different aspects of predicting movements.

1. The first model is known to be a gold standard in financial use cases, LSTM. We designed it to have two layers with 50 units each, where each unit is essentially a "mind". One of the layers transfers all the processed information to the next training sequence whereas the second one transfers only the processed result. In both cases, we dropped 20% of the connections to prevent the model from memorizing patterns and to make it more generalizable. We also selected the "Adam" optimizer to adapt and learn at a conservative rate of 0.1%. We trained the model on the same data using 50 epochs, meaning it reviews and refines itself 50 consecutive times.
2. The second model we designed was a Gated Recurrent Unit (GRU) model. We designed it to have 4 layers. One layer with 100 units to give it more analytical horsepower, one with 50 units to refine itself more flexibly, then two dense layers with 25 and 1 unit respectively to serve as final decision-makers. We increased the dropout rate to 30% and 20% for the first two layers as they learn from a wider dataset. To train it even more conservatively, we set a learning rate of 0.005% and also trained it on 60 epochs of data.
3. Finally, the third model was a Simple RNN model. We designed this one to have 4 layers as well. One layer with 80 units, the next one with 40 and two dense layers of 20 and 1 units. This was done with the intention of focusing on more recent records and patterns. We added somewhat more moderate dropout rates of 15 to 25% and gave it increased learning rate to 0.02% to improve training speed. Though we gave it 64 epochs to train itself on, the model achieved optimum performance with only 40 epochs.

Each model represents a different use case for our stakeholders. LSTM is tuned for comprehensive pattern recognition and GRU is best for balanced performance. Finally, Simple RNN is best for high-speed forecasting.